

```
In [ ]: # Mid Term Practical Examination
# Assignment No.03
# Deokar Shraddha Sunil
# TY - B - 153
```

```
In [2]: # Import required libraries

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
In [3]: # Load dataset

df = pd.read_csv("Mobile Price Classification.csv")

print("Dataset Shape:", df.shape)

df.head()
```

Dataset Shape: (2000, 21)

```
Out[3]:
```

	battery_power	blue	clock_speed	dual_sim	fc	four_g	int_memory	m_dep	mobile_w
0	842	0	2.2	0	1	0	7	0.6	188
1	1021	1	0.5	1	0	1	53	0.7	136
2	563	1	0.5	1	2	1	41	0.9	141
3	615	1	2.5	0	0	0	10	0.8	137
4	1821	1	1.2	0	13	1	44	0.6	141

5 rows × 21 columns



```
In [4]: # Dataset information

df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   battery_power         2000 non-null   int64
 1   blue                  2000 non-null   int64
 2   clock_speed           2000 non-null   float64
 3   dual_sim              2000 non-null   int64
 4   fc                    2000 non-null   int64
 5   four_g                2000 non-null   int64
 6   int_memory            2000 non-null   int64
 7   m_dep                 2000 non-null   float64
 8   mobile_wt             2000 non-null   int64
 9   n_cores               2000 non-null   int64
10   pc                    2000 non-null   int64
11   px_height              2000 non-null   int64
12   px_width              2000 non-null   int64
13   ram                   2000 non-null   int64
14   sc_h                  2000 non-null   int64
15   sc_w                  2000 non-null   int64
16   talk_time             2000 non-null   int64
17   three_g               2000 non-null   int64
18   touch_screen          2000 non-null   int64
19   wifi                  2000 non-null   int64
20   price_range           2000 non-null   int64
dtypes: float64(2), int64(19)
memory usage: 328.3 KB

```

```

In [5]: plt.figure(figsize=(6,4))

sns.countplot(x="price_range", data=df)

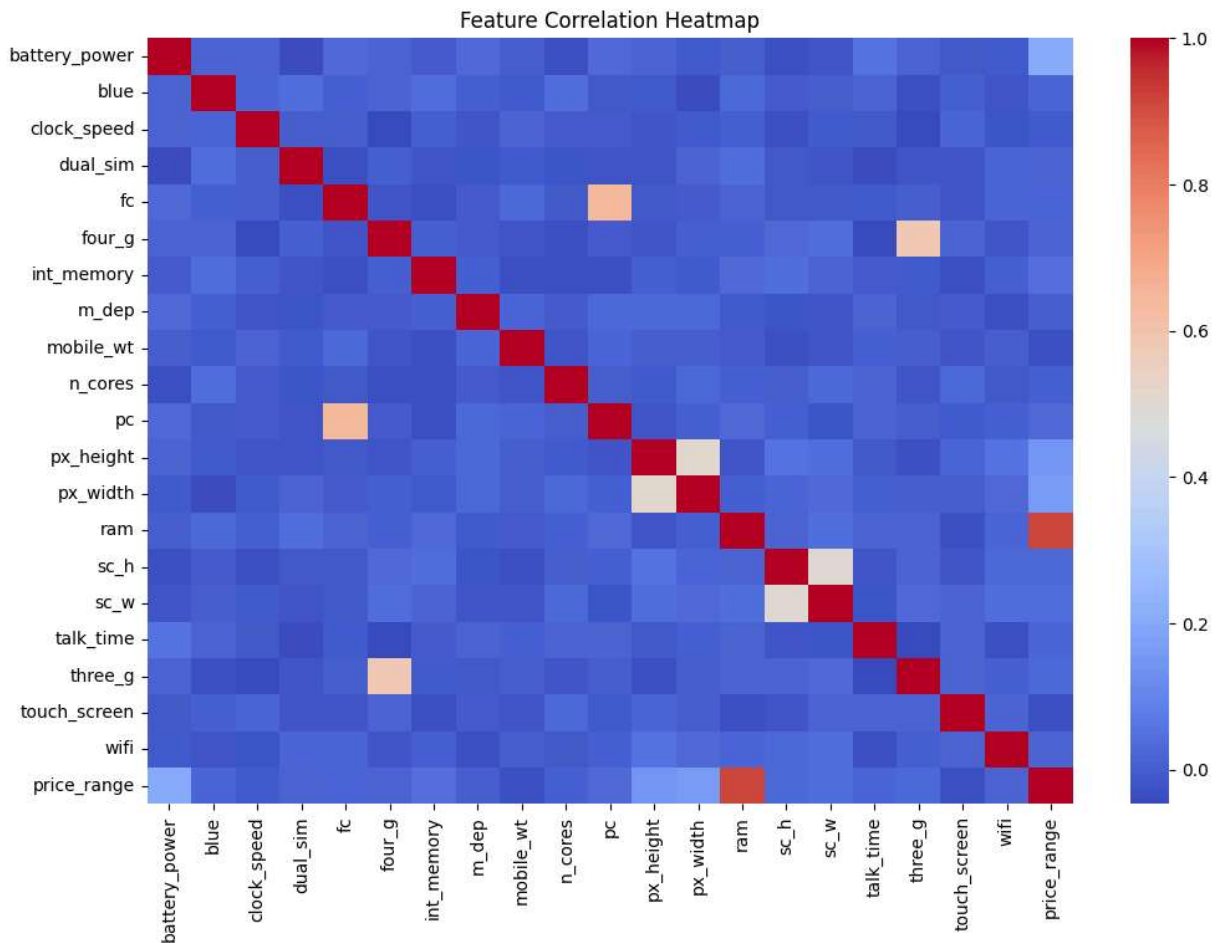
plt.title("Distribution of Price Range")

plt.show()

```



```
In [6]: plt.figure(figsize=(12,8))  
  
sns.heatmap(df.corr(), cmap="coolwarm")  
  
plt.title("Feature Correlation Heatmap")  
  
plt.show()
```



In [7]: *# Separate features and target*

```
X = df.drop("price_range", axis=1)

y = df["price_range"]
```

In [8]:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

Training Data Shape: (1600, 20)

Testing Data Shape: (400, 20)

In [9]:

```
xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=6,
    objective="multi:softmax",
    num_class=4,
    eval_metric="mlogloss"
)
```

```
xgb_model.fit(X_train, y_train)
```

Out[9]:

▼ XGBClassifier ⓘ ?

► Parameters

```
In [10]: y_pred_xgb = xgb_model.predict(X_test)
```

```
In [11]: xgb_accuracy = accuracy_score(y_test, y_pred_xgb)

print("XGBoost Accuracy:", xgb_accuracy)

print("\nClassification Report:\n")

print(classification_report(y_test, y_pred_xgb))
```

XGBoost Accuracy: 0.9

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.95	0.95	105
1	0.85	0.93	0.89	91
2	0.85	0.82	0.83	92
3	0.93	0.89	0.91	112
accuracy			0.90	400
macro avg	0.90	0.90	0.90	400
weighted avg	0.90	0.90	0.90	400

```
In [12]: cm = confusion_matrix(y_test, y_pred_xgb)

plt.figure(figsize=(6,5))

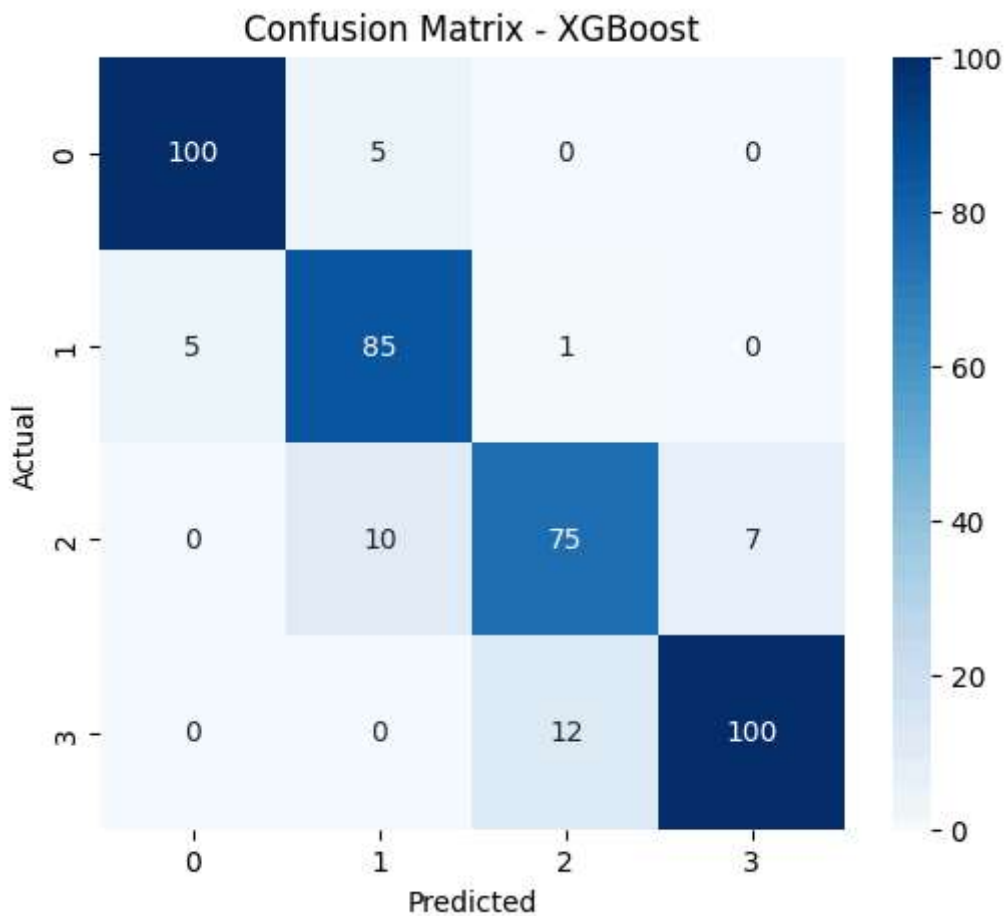
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.title("Confusion Matrix - XGBoost")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```



```
In [13]: cv_scores = cross_val_score(xgb_model, X, y, cv=5)

print("Cross Validation Scores:", cv_scores)

print("Average CV Score:", cv_scores.mean())
```

Cross Validation Scores: [0.9175 0.925 0.9075 0.9025 0.895]
Average CV Score: 0.9094999999999999

```
In [14]: lgb_model = LGBMClassifier(
            n_estimators=100,
            learning_rate=0.1
        )

lgb_model.fit(X_train, y_train)

y_pred_lgb = lgb_model.predict(X_test)

lgb_accuracy = accuracy_score(y_test, y_pred_lgb)

print("LightGBM Accuracy:", lgb_accuracy)
```

```
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.000317 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 1361
[LightGBM] [Info] Number of data points in the train set: 1600, number of used features: 20
[LightGBM] [Info] Start training from score -1.398873
[LightGBM] [Info] Start training from score -1.364044
[LightGBM] [Info] Start training from score -1.366492
[LightGBM] [Info] Start training from score -1.416754
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
LightGBM Accuracy: 0.9025
```

```
In [15]: models = ["XGBoost", "LightGBM"]

accuracies = [xgb_accuracy, lgb_accuracy]

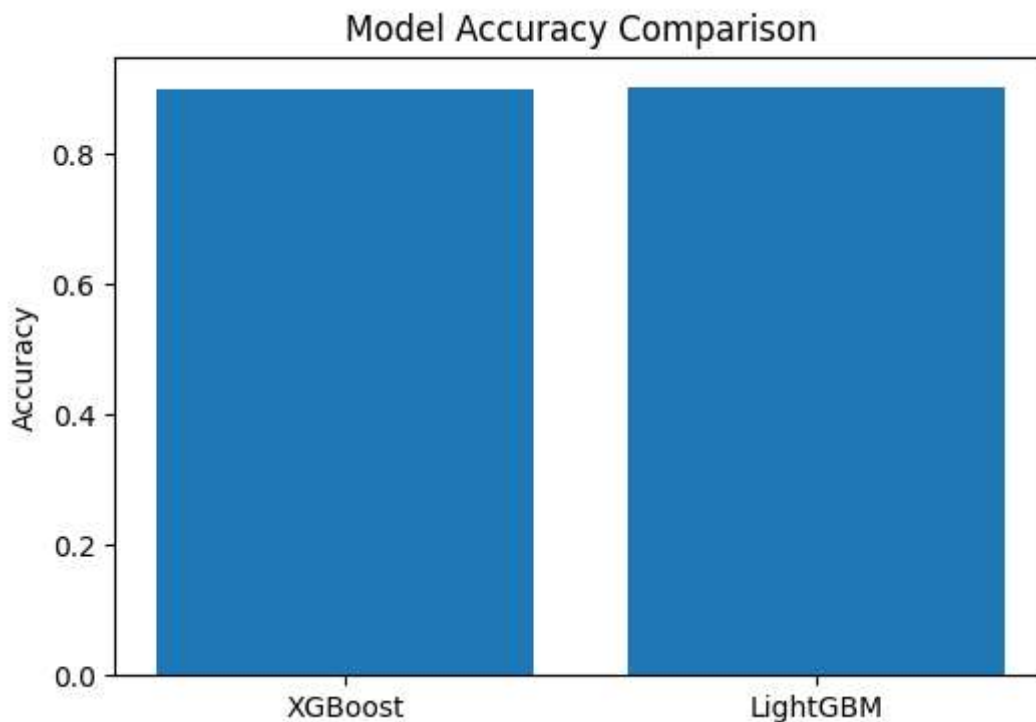
plt.figure(figsize=(6,4))

plt.bar(models, accuracies)

plt.title("Model Accuracy Comparison")

plt.ylabel("Accuracy")

plt.show()
```



```
In [16]: importance = xgb_model.feature_importances_

features = X.columns

plt.figure(figsize=(10,6))
```

```
sns.barplot(x=importance, y=features)

plt.title("Feature Importance (XGBoost)")

plt.show()
```

